

Evaluation Chapter: Hypothesis-Driven Evaluation of n8n Workflow Generation and Validation

1 Evaluation Objective

The evaluation is structured around three hypotheses. Together, they cover three complementary levels of analysis:

- the robustness of generated workflows as executable systems,
- the quality improvement obtained from a specialized workflow-generation skill,
- the validation of individual LLM-based building blocks inside a workflow.

This structure moves the evaluation away from a purely validator-centric comparison and towards a workflow-centric and hypothesis-driven study design.

2 Overview of the Three Hypotheses

2.1 Hypothesis A: Workflow Robustness Through Test Runs

Hypothesis A: Generated n8n workflows can be evaluated meaningfully through repeated end-to-end test runs, and their success rate differs systematically across workflow complexity levels.

The focus is not on validating isolated process fragments. Instead, the evaluation measures how often a generated workflow actually works under realistic or simulated execution conditions.

2.2 Hypothesis B: Skill-Supported Generation Improves Workflow Quality

Hypothesis B: A specialized workflow-generation skill with integrated self-correction produces runnable and correctable workflows more reliably than a general baseline prompting approach.

This hypothesis evaluates the engineering contribution of the thesis. In particular, it examines whether a dedicated skill, potentially including targeted TypeScript adaptations for the frontend or supporting tooling, improves generation quality and repair behavior.

2.3 Hypothesis C: Individual LLM Building Blocks Can Be Validated with Different Measures

Hypothesis C: For individual LLM building blocks inside workflows, combined validation methods detect errors more reliably than any single validation method alone.

This hypothesis focuses on LLM nodes as special workflow elements that require richer validation methods than ordinary structural checks.

3 Overall Evaluation Design

The evaluation is divided into three coordinated experiments:

1. an execution-based experiment for complete workflows,
2. a generation-quality experiment for the workflow-construction skill,
3. a component-level validation experiment for LLM nodes.

All three experiments are based on n8n workflows, but they differ in unit of analysis:

- in Hypothesis A, the unit of analysis is the entire workflow run,
- in Hypothesis B, the unit of analysis is the workflow generation attempt,
- in Hypothesis C, the unit of analysis is the individual LLM node or LLM task.

4 Dataset and Artifact Design

A realistic study can be based on approximately 100 business workflow descriptions distributed across several classes:

- 40 sequential business workflows,
- 30 workflows with branching and parallelism,
- 20 workflows with exception handling or approval logic,
- 10 workflows containing 1-2 explicit LLM nodes.

For each workflow description, several artifacts are maintained:

- a natural-language process description,
- a generated workflow candidate,
- where needed, a reference workflow,
- test inputs and expected outputs,
- execution logs,
- error labels and complexity metadata.

The same workflow pool can be reused across the three hypotheses, but each hypothesis emphasizes a different view on the artifacts.

5 Evaluation of Hypothesis A

5.1 Goal

Hypothesis A evaluates how often generated workflows succeed during repeated execution.

The main question is: *How frequently does a generated workflow actually complete successfully under realistic test conditions?*

5.2 Why Test Runs Instead of Pure Static Validation

Static inspection alone does not capture whether a workflow can execute correctly in practice. A workflow may look structurally plausible while still failing during runtime because of:

- malformed expressions,
- incorrect routing of data between nodes,
- missing fields in intermediate payloads,
- brittle assumptions about external systems,
- unstable LLM outputs.

Therefore, Hypothesis A relies on repeated test runs as the main evidence source.

5.3 Test Setup

For each workflow:

1. prepare a fixed test input,
2. run the workflow repeatedly under the same or slightly varied conditions,
3. collect execution results, logs, and failure points,
4. compare the observed outcome with the expected business outcome.

Wherever possible, external systems should be mocked or simulated. This includes:

- email nodes,
- HTTP endpoints,
- databases,
- ERP or CRM integrations.

This keeps the test runs reproducible and avoids side effects.

5.4 Metrics for Hypothesis A

The key metrics are:

- workflow success rate,
- failure rate per workflow,
- failure rate per node type,
- repeatability across multiple runs,
- time to first successful run,
- average runtime per successful execution.

5.5 Complexity Analysis

To test the second part of Hypothesis A, workflow complexity is operationalized through:

- number of nodes,
- number of edges,
- number of branches,
- number of merges,
- control-flow depth,
- number of external integrations,
- number of LLM nodes.

The analysis then compares execution success rates across low, medium, and high complexity groups.

5.6 Expected Result Pattern

It is expected that simple workflows show substantially higher success rates than complex workflows. The evaluation is therefore not aimed at proving that workflows either always work or always fail, but at identifying where robustness starts to break down.

6 Evaluation of Hypothesis B

6.1 Goal

Hypothesis B evaluates whether a dedicated workflow-generation skill improves workflow construction compared with a more generic baseline prompting approach.

6.2 Compared Conditions

At least the following conditions should be compared:

- baseline prompting without a specialized skill,
- specialized skill for workflow generation,
- specialized skill plus explicit self-correction or repair loop.

Optionally, a fourth condition can be added:

- specialized skill with targeted TypeScript adaptation support for connected frontend or tooling changes.

6.3 Unit of Analysis

The unit of analysis is one generation attempt. Each attempt receives the same business requirement, but a different generation setup.

Each attempt produces:

- a workflow JSON candidate,
- optional repair iterations,
- prompt and trace data,
- an execution outcome from a follow-up run.

6.4 Metrics for Hypothesis B

The most relevant metrics are:

- proportion of syntactically valid workflow JSON,
- proportion of importable workflows,
- proportion of runnable workflows,
- number of repair iterations,
- human post-editing effort,
- time per successful workflow,
- token cost per successful workflow.

6.5 Error Categories for Hypothesis B

The generation outcomes can be labeled using categories such as:

- JSON syntax error,
- invalid node configuration,
- wrong control-flow structure,
- missing required business step,
- incorrect use of an LLM node,
- frontend or TypeScript integration issue,
- execution failure despite syntactically valid workflow.

6.6 Expected Result Pattern

The expected non-trivial result is that a specialized skill does not merely improve textual quality, but measurably increases the fraction of runnable workflows and reduces the human correction burden.

7 Evaluation of Hypothesis C

7.1 Goal

Hypothesis C investigates how individual LLM building blocks inside workflows can be validated.

This evaluation does not attempt to validate the full philosophical truth of every generated answer. Instead, it focuses on whether an LLM node behaves plausibly, safely, and consistently relative to its role in the workflow.

7.2 Why a Separate LLM Evaluation Is Needed

LLM nodes differ from classical nodes because:

- their outputs are not fully deterministic,
- their quality depends on prompts and context,
- many of their errors are semantic rather than structural,
- the same task may look correct structurally but still be wrong in content.

7.3 Validation Method Families

The evaluated methods are grouped into several categories.

Category	Technology	Short description
Basis	Embedding Similarity	Semantic similarity between prompt, context, and output
Basis	NLI	Checks whether an output statement logically follows from the available context
Model-based	LLM-as-a-Judge	Stronger LLM directly evaluates the output against criteria
Model-based	Self-Consistency Sampling	Multiple generations are compared for stability
Model-based	Debate / Multi-Agent	Several models critique each other or defend alternative judgments
Knowledge-based	Claim Extraction + Verification	Output is decomposed into claims and each claim is verified separately
Knowledge-based	Retrieval-Augmented Validation	External evidence is retrieved and compared to the output
Uncertainty-based	Uncertainty Scoring	Confidence or uncertainty signals are used as warning indicators
Rule-based	Programmatic Constraints	Format, schema, business rules, and structural constraints are checked deterministically
Stress-based	Adversarial Testing	Inputs are generated systematically to provoke failures

Table 1: Validation methods for individual LLM building blocks

7.4 Experimental Design for Hypothesis C

For each LLM task:

1. define the intended purpose of the LLM node,
2. prepare prompt, context, and expected evaluation criteria,
3. generate one or more LLM outputs,
4. apply several validation methods to the same output,
5. compare which methods detect which error types.

Representative LLM task types may include:

- summarization,
- classification,
- extraction,
- draft generation,

- recommendation,
- routing support inside a workflow.

7.5 Error Types for Hypothesis C

Relevant error classes include:

- factual error,
- unsupported claim,
- missing critical information,
- unsafe recommendation,
- wrong output format,
- unstable output across repeated generations,
- mismatch between the node’s intended purpose and actual output.

7.6 Metrics for Hypothesis C

Depending on the method, the following metrics are relevant:

- Precision,
- Recall,
- F1,
- false negative rate for critical errors,
- agreement with human judgment,
- runtime,
- token cost.

7.7 Expected Result Pattern

It is not expected that one single validation measure dominates in all situations. More likely:

- rule-based constraints are strong for format and safety guards,
- embedding and NLI methods are useful for lightweight semantic checks,
- LLM-as-a-Judge is stronger for nuanced semantic evaluation,
- retrieval-based and claim-based methods are helpful where factual grounding matters,
- combined approaches perform best overall.

8 Cross-Hypothesis Metrics and Result Presentation

Although each hypothesis has its own main metrics, several output formats are shared across the chapter.

8.1 Tables

- success rates for workflow runs,
- generation quality per skill condition,
- validation performance per LLM method,
- runtime and token-cost comparisons.

8.2 Charts

- bar charts for workflow success rates,
- line charts over increasing workflow complexity,
- boxplots for runtime and token costs,
- heatmaps for validation methods by error classes.

8.3 Case-Based Analysis

In addition to quantitative summaries, selected cases should be documented qualitatively:

- highly successful workflow generations,
- workflows that are syntactically valid but fail at runtime,
- cases where the skill repairs a failure successfully,
- LLM-node outputs that pass simple checks but fail stronger validation,
- adversarial examples revealing hidden weaknesses.

9 Conclusion of the Evaluation Setup

The proposed setup provides a coherent evaluation strategy across three levels:

- **macro level:** Does the whole workflow run successfully?
- **meso level:** Does the specialized skill improve workflow generation and repair?
- **micro level:** How can individual LLM building blocks be validated?

This structure is expected to produce heterogeneous and therefore scientifically meaningful results. It avoids reducing the thesis to a single validator benchmark and instead frames workflow execution, workflow construction, and LLM-node validation as distinct but connected research contributions.